Day 37 — Line-by-Line Syntax Explanation

day37_loss_functions_optimizers.py, segment by segment, line by line. Pure Python/NumPy syntax focus, with the math each chunk implements, a system diagram of how it works, and real (rerun, not fabricated) graphs showing the code+math turned into a visualization.

MD Mutasim Billah | 52-week Data Science → ML/AI roadmap | Week 6, Day 2 reference

This document walks the script top to bottom in the order it actually runs. Each segment matches one numbered section of the script. Wherever a code chunk implements a real formula (MSE/BCE loss, gradient descent, Momentum, Adam, backpropagation), a purple **MATH** box shows that formula immediately before the code, followed by the syntax explanation. After each major segment, a green **SYSTEM DIAGRAM** shows how that math and code fit together as a working system — including the training work cycle, and the script's own real output graphs.

MATH box legend: × = matrix multiply / times W1, b1 = weights/bias of layer 1, etc. X^T = transpose of X
sum[...] = add up over all examples m = number of training examples t = optimizer step count

Segment 0 — Imports and Global Setup (lines 31–37)

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons

np.random.seed(42)
```

Identical setup pattern to Day 36: **np** for all array math, **matplotlib.use("Agg")** before importing **pyplot** so charts render straight to PNG files instead of a GUI window, **make_moons** imported directly for the real-network test later, and **np.random.seed(42)** fixing every subsequent "random" NumPy call so this script's numbers are exactly reproducible.

Segment 1 — Loss Functions (lines 44–67)

MATH

$$MSE(pred, target) = mean((pred - target)^2)$$

```
def mse_loss(pred, target):
    return np.mean((pred - target) ** 2)
```

A plain function taking two NumPy arrays. **pred - target** subtracts elementwise; **** 2** squares every element; **np.mean(...)** averages all of them down to one number. No loop anywhere — three vectorized operations chained on one line.

MATH

$$BCE(pred, target) = -mean[target \times log(pred) + (1-target) \times log(1-pred)]$$

(pred is clipped to [eps, 1-eps] first so log(0) never happens)

```
def bce_loss(pred, target, eps=1e-9):
    pred = np.clip(pred, eps, 1 - eps)
    return -np.mean(target * np.log(pred) + (1 - target) * np.log(1 - pred))
```

eps=1e-9 is a default argument. **np.clip(pred, eps, 1 - eps)** forces every element of **pred** into the range [eps, 1-eps] — anything below **eps** gets pushed up to it, anything above **1-eps** gets pushed down — which prevents **np.log(0)** (negative infinity) from ever occurring on the next line, exactly like Day 36's TinyNet. The return line is the binary cross-entropy formula in one vectorized expression, negated at the front.

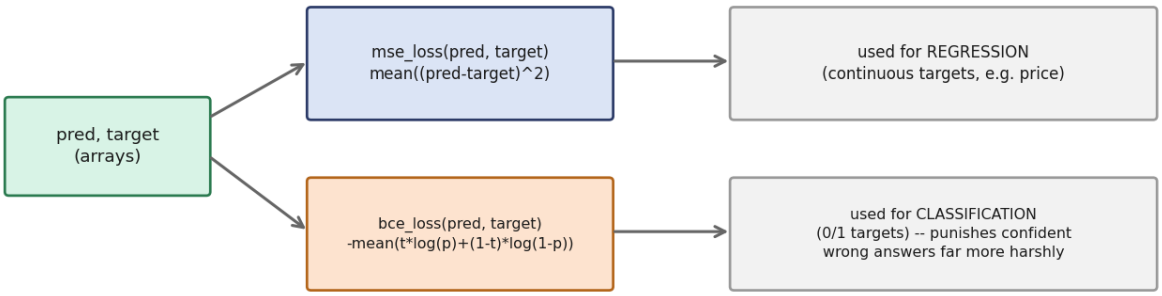
```
pred_good = np.array([0.9, 0.05])
pred_bad = np.array([0.1, 0.95])
target = np.array([1, 0])
```

Two hand-picked prediction arrays representing the same two true labels (**target = [1, 0]**) — one confidently correct (**pred_good**: 0.9 for class 1, 0.05 for class 0), one confidently wrong (**pred_bad**: only 0.1 for the true class-1 example, 0.95 for the true class-0 example) — set up specifically to make MSE and BCE disagree on how much to penalize each.

```
mse_g = mse_loss(pred_good, target)
bce_g = bce_loss(pred_good, target)
print(f"  MSE: {mse_g:.4f}   BCE: {bce_g:.4f}")
```

Both loss functions called inline inside an f-string, each formatted to 4 decimal places with **{...:.4f}** — the same format-spec syntax seen throughout Day 36's activation-function printing.

SYSTEM DIAGRAM



Both formulas from this segment as one system: the same **(pred, target)** pair can flow into either loss function, and the two produce very different numbers on the same bad prediction — that gap is the entire reason BCE, not MSE, is used to train classifiers.

Segments 2–4 — Gradient Descent / Momentum / Adam on a Toy Surface (lines 74–161)

MATH

$$f(x, y) = 0.1 \times x^2 + 2 \times y^2$$

```
def toy_surface(xy):
    x, y = xy
    return 0.1 * x ** 2 + 2 * y ** 2
```

x, y = xy unpacks a 2-element array into two separate scalars in one line (tuple/array unpacking). The return line evaluates a fixed mathematical surface at that point — deliberately much steeper in **y** than **x** (coefficient 2 vs 0.1), which is exactly what makes plain gradient descent zigzag.

MATH

$$df/dx = 0.2 \times x \quad (\text{derivative of } 0.1x^2)$$
$$df/dy = 4 \times y \quad (\text{derivative of } 2y^2)$$
$$gradient(x, y) = [\ df/dx, df/dy\ ]$$

```
def toy_gradient(xy):
    x, y = xy
    return np.array([0.2 * x, 4 * y])
```

The hand-derived gradient of **toy_surface** — calculus done by hand and typed directly into code, no autograd involved. Returns a 2-element array: one partial derivative per input dimension, packaged the same shape as the point it was evaluated at.

MATH

$$xy = xy - lr \times gradient(xy) \quad (\text{repeated once per step})$$

```
def run_sgd(start, lr, steps=40):
    xy = np.array(start, dtype=float)
    path = [xy.copy()]
    for _ in range(steps):
        xy = xy - lr * toy_gradient(xy)
        path.append(xy.copy())
    return np.array(path)
```

np.array(start, dtype=float) converts the starting point (a plain Python list like **[-8, -4]**) into a float NumPy array — **dtype=float** matters because in-place updates on an int array would silently truncate fractional steps. **path = [xy.copy()]** starts a list recording every visited point; **.copy()** is essential here — without it, **path** would store references to the same array object, and every later mutation of **xy** would retroactively change every entry already appended. Each loop iteration does one plain gradient-descent step and records the new position.

MATH

v = beta × v + (1 - beta) × g (velocity: exponential moving average of gradients)
xy = xy - lr × v

```
    def run_momentum(start, lr, beta=0.7, steps=40):
        xy = np.array(start, dtype=float)
        v = np.zeros(2)
        path = [xy.copy()]
        for _ in range(steps):
            g = toy_gradient(xy)
            v = beta * v + (1 - beta) * g
            xy = xy - lr * v
            path.append(xy.copy())
        return np.array(path)
```

v = np.zeros(2) initializes a 2D "velocity" vector at zero, separate from the position **xy**. Each step, **v** is updated to an **exponential moving average** of the gradient — mostly the old velocity (**beta * v**), blended with a bit of the new gradient (**(1 - beta) * g**) — and the position moves using **v** instead of the raw gradient directly, which is what smooths out the zigzag.

```
    def run_adam(start, lr, beta1=0.9, beta2=0.999, steps=40, eps=1e-8):
        xy = np.array(start, dtype=float)
        m = np.zeros(2)
        v = np.zeros(2)
        path = [xy.copy()]
        for t in range(1, steps + 1):
```

Two accumulators this time: **m** (mirrors Momentum's velocity — moving average of the raw gradient) and **v** (a second moving average, of the *squared* gradient, tracking how large recent gradients have been). **for t in range(1, steps + 1):** starts counting from **1**, not **0** — Adam's bias-correction formula divides by a term involving **beta ** t**, and starting at **t=0** would make that correction divide by zero on the first step.

MATH

m = beta1 × m + (1-beta1) × g (1st moment: mean of g)
v = beta2 × v + (1-beta2) × g^2 (2nd moment: mean of g^2)
m_hat = m / (1 - $beta1^t$) v_hat = v / (1 - $beta2^t$) (bias correction)
xy = xy - lr × m_hat / (sqrt(v_hat) + eps)

```
            g = toy_gradient(xy)
            m = beta1 * m + (1 - beta1) * g
            v = beta2 * v + (1 - beta2) * (g ** 2)
            m_hat = m / (1 - beta1 ** t)
            v_hat = v / (1 - beta2 ** t)
            xy = xy - lr * m_hat / (np.sqrt(v_hat) + eps)
            path.append(xy.copy())
        return np.array(path)
```

m and **v** update the same way Momentum's **v** does, just with two different decay rates (**beta1=0.9**, **beta2=0.999**) and two different signals (**g** vs **g ** 2**). **m_hat**/**v_hat** are **bias-corrected** versions — early on, when **t** is small, **m** and **v** are biased toward zero (they started at zero and haven't accumulated much yet);

dividing by **(1 - beta ** t)** compensates for that, and the correction fades out as **t** grows since **beta ** t -> 0**. The final update divides the step by **sqrt(v_hat)** — this is the "adaptive" part: parameters with consistently large gradients get automatically smaller steps, and vice versa. **eps** in the denominator just guards against division by zero.

```
start = [-8, -4]
path_sgd = run_sgd(start, lr=0.46)
path_mom = run_momentum(start, lr=0.46, beta=0.7)
path_adam = run_adam(start, lr=0.30)
```

All three optimizers race from the same starting point with individually tuned learning rates, each returning its own full path of visited (x, y) points as a NumPy array.

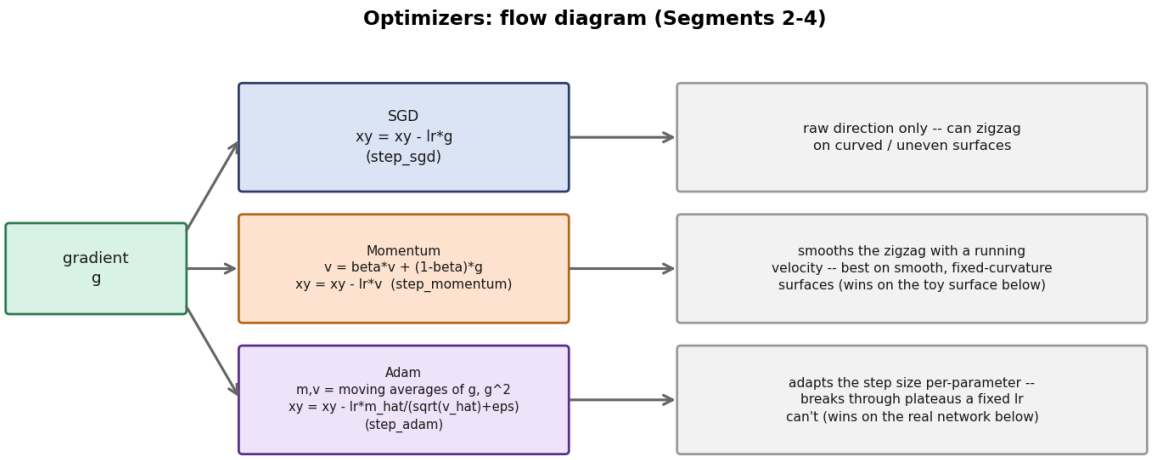
```
final_sgd = toy_surface(path_sgd[-1])
print(f"Plain SGD (lr=0.46): final loss={final_sgd:.5f}")
```

path_sgd[-1] — negative indexing grabs the **last** element of the array, i.e. where that optimizer ended up after all 40 steps. Feeding it back into **toy_surface(...)** reports how low a loss value that final position corresponds to, formatted to 5 decimal places.

```
fig, axes = plt.subplots(1, 3, figsize=(13, 4.2))
sgd_title = f"Plain SGD, lr=0.46\nfinal loss="\
    f"{toy_surface(path_sgd[-1]):.4f}"
panels = [
    (axes[0], path_sgd, sgd_title),
    ...
]
for ax, path, title in panels:
    ax.contour(XG, YG, ZG, levels=15, colors="#D3D1C7", linewidths=0.7)
    ax.plot(path[:, 0], path[:, 1], "-o", color="#7A4EC9", markersize=2.5)
    ax.plot(0, 0, "*", color="#D85A30", markersize=14)
```

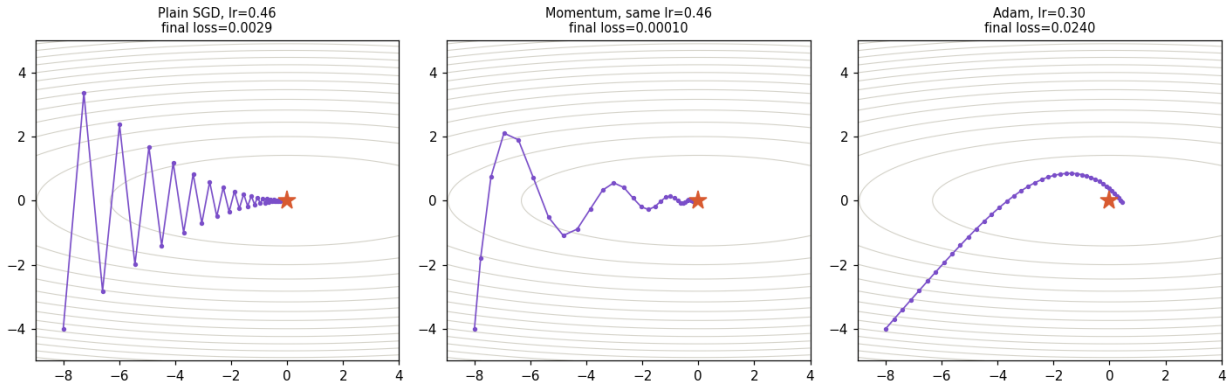
Same **panels**-list-plus-loop pattern from Day 36's decision boundary plot. **ax.contour(XG, YG, ZG, levels=15, ...)** draws 15 elevation lines of the toy surface — the same idea as a topographic map. **ax.plot(path[:, 0], path[:, 1], "-o", ...)** plots the optimizer's actual trajectory: column 0 of **path** as x-coordinates, column 1 as y-coordinates, connected by a line with circular markers at each step. **ax.plot(0, 0, "*", ...)** marks the true minimum with a star so you can see how close each path gets.

SYSTEM DIAGRAM



The three optimizer formulas from this segment, side by side: same input gradient, three different update rules, three different tradeoffs — and (spoiler, from the real printed numbers) Momentum wins on this particular smooth toy surface.

REAL GRAPH (script's own output)



Not a mockup — this is **optimizer_paths.png**, this exact script rerun fresh. Each panel is one call to **run_sgd/run_momentum/run_adam** plotted with **ax.contour** + **ax.plot** as covered above. Watch the SGD panel zigzag between the contour lines — that visible back-and-forth *is* the plain-gradient-descent problem this whole segment exists to solve.

Segment 5 — The Real Test: SGD vs Momentum vs Adam (lines 168–273)

```
class Net:
    """Day 36's TinyNet, generalized with a pluggable optimizer."""

    def __init__(self, n_in, n_hidden, seed=42):
        rng = np.random.RandomState(seed)
        self.W1 = rng.randn(n_in, n_hidden) * 0.5
        self.b1 = np.zeros(n_hidden)
        self.W2 = rng.randn(n_hidden, 1) * 0.5
        self.b2 = np.zeros(1)
```

np.random.RandomState(seed) creates an **independent** random number generator, separate from NumPy's global **np.random** state — using **rng.randn(...)** instead of **np.random.randn(...)** means this network's initial weights don't depend on how many other random calls happened earlier in the script, which is exactly why all three optimizer runs below start from *identical* weights — a fair comparison.

```
        keys = ["W1", "b1", "W2", "b2"]
        self.m = {k: np.zeros_like(getattr(self, k)) for k in keys}
        self.v = {k: np.zeros_like(getattr(self, k)) for k in keys}
        self.t = 0
```

A **dict comprehension**: for each parameter name in the list **["W1", "b1", "W2", "b2"]**, build one dictionary entry. **getattr(self, k)** looks up the attribute named by the string **k** — e.g. **getattr(self, "W1")** is the same thing as **self.W1**, just accessed dynamically by name instead of hard-coded. **np.zeros_like(...)** creates an array of zeros the exact same shape as whatever it's given. The result: **self.m** and **self.v** are dictionaries holding one Adam accumulator per parameter, all pre-shaped and zeroed, without writing out 4 separate lines by hand. **self.t = 0** is the shared step counter Adam needs.

MATH

$$z1 = X \times W1 + b1 \quad a1 = sigmoid(z1)$$
$$z2 = a1 \times W2 + b2 \quad a2 = sigmoid(z2) = y\_pred$$

```
    def forward(self, X):
        self.z1 = X @ self.W1 + self.b1
        self.a1 = sigmoid(self.z1)
        self.z2 = self.a1 @ self.W2 + self.b2
        self.a2 = sigmoid(self.z2)
        return self.a2
```

Character-for-character identical logic to Day 36's **TinyNet.forward()** — same 4-line forward pass, same reason for caching **z1**/**a1**/**z2**/**a2** as instance attributes for **grads()** to reuse.

MATH

$dz2 = a2 - y \quad dW2 = (1/m) \times a1^T \times dz2 \quad db2 = mean(dz2)$
$da1 = dz2 \times W2^T \quad dz1 = da1 \times a1 \times (1-a1)$
$dW1 = (1/m) \times X^T \times dz1 \quad db1 = mean(dz1)$

```
        def grads(self, X, y_col):
            m = X.shape[0]
            dz2 = self.a2 - y_col
            dW2 = self.a1.T @ dz2 / m
            db2 = dz2.mean(axis=0)
            da1 = dz2 @ self.W2.T
            dz1 = da1 * self.a1 * (1 - self.a1)
            dW1 = X.T @ dz1 / m
            db1 = dz1.mean(axis=0)
            return {"W1": dW1, "b1": db1, "W2": dW2, "b2": db2}
```

The same backpropagation math as Day 36's **TinyNet.backward()**, with one structural difference: instead of updating **self.W1** etc. directly (which locked the update rule into one fixed formula), **grads()** only **computes and returns** the gradients, packaged into a dictionary keyed by parameter name. This separation is exactly what makes the optimizer **pluggable** — the same gradients get handed off to whichever **step_*** method the caller chooses.

MATH

for each parameter: param = param - lr × gradient

```
        def step_sgd(self, g, lr):
            for k in g:
                setattr(self, k, getattr(self, k) - lr * g[k])
```

for k in g: iterates over a dictionary's **keys** by default — so **k** takes the values **"W1"**, **"b1"**, **"W2"**, **"b2"** in turn. **getattr(self, k) - lr * g[k]** computes the updated value; **setattr(self, k, ...)** writes it back onto the matching attribute — together, **getattr**/**setattr** let one 2-line loop update all 4 parameters generically, instead of writing **self.W1 -= lr*dW1**, **self.b1 -= lr*db1**, etc. by hand as Day 36 did.

MATH

for each parameter: m = beta × m + (1-beta) × gradient
param = param - lr × m

```
        def step_momentum(self, g, lr, beta=0.9):
            for k in g:
                self.m[k] = beta * self.m[k] + (1 - beta) * g[k]
                setattr(self, k, getattr(self, k) - lr * self.m[k])
```

Same generic **getattr**/**setattr** loop, but each parameter now has its own persistent velocity stored in **self.m[k]** (reused across every call to **step_momentum**, unlike the toy surface's **run_momentum** where **v** was a single local variable for a single 2D point).

MATH

$t = t + 1$ *(once per step_adam call)*
for each parameter: $m = beta1 \times m + (1-beta1) \times g$ $\quad v = beta2 \times v + (1-beta2) \times g^2$
$m\_hat = m/(1-beta1^t) \quad v\_hat = v/(1-beta2^t)$
$param = param - lr \times m\_hat / (sqrt(v\_hat) + eps)$

```
        def step_adam(self, g, lr, beta1=0.9, beta2=0.999, eps=1e-8):
            self.t += 1
            for k in g:
                self.m[k] = beta1 * self.m[k] + (1 - beta1) * g[k]
                self.v[k] = beta2 * self.v[k] + (1 - beta2) * (g[k] ** 2)
                m_hat = self.m[k] / (1 - beta1 ** self.t)
                v_hat = self.v[k] / (1 - beta2 ** self.t)
                new_val = getattr(self, k) - lr * m_hat / (np.sqrt(v_hat) + eps)
                setattr(self, k, new_val)
```

self.t += 1 increments the shared step counter **once per call** to **step_adam** — not once per parameter — so all 4 parameters use the same **t** for bias correction on a given epoch. Otherwise this is the exact same Adam math as **run_adam** on the toy surface, just applied per-key across the dictionary instead of to one flat array.

```
    def train(X, y, optimizer, lr, epochs=1000):
        y_col = y.reshape(-1, 1)
        net = Net(n_in=2, n_hidden=8, seed=42)
        losses = []
        for _ in range(epochs):
            pred = net.forward(X)
            losses.append(bce_loss(pred, y_col))
            g = net.grads(X, y_col)
            if optimizer == "sgd":
                net.step_sgd(g, lr)
            elif optimizer == "momentum":
                net.step_momentum(g, lr)
            elif optimizer == "adam":
                net.step_adam(g, lr)
```

This is the segment's entire work cycle in one function: forward pass, record loss, compute gradients, then an **if/elif** chain picking which optimizer's **step_*** method to call based on the **optimizer** string argument — the same 4 gradients from **net.grads()** feed into whichever update rule was requested, which is the whole point of separating gradient computation from the optimizer step.

MATH

accuracy = (1/m) × sum over i of [$y\_pred_i == y_i$ *]*

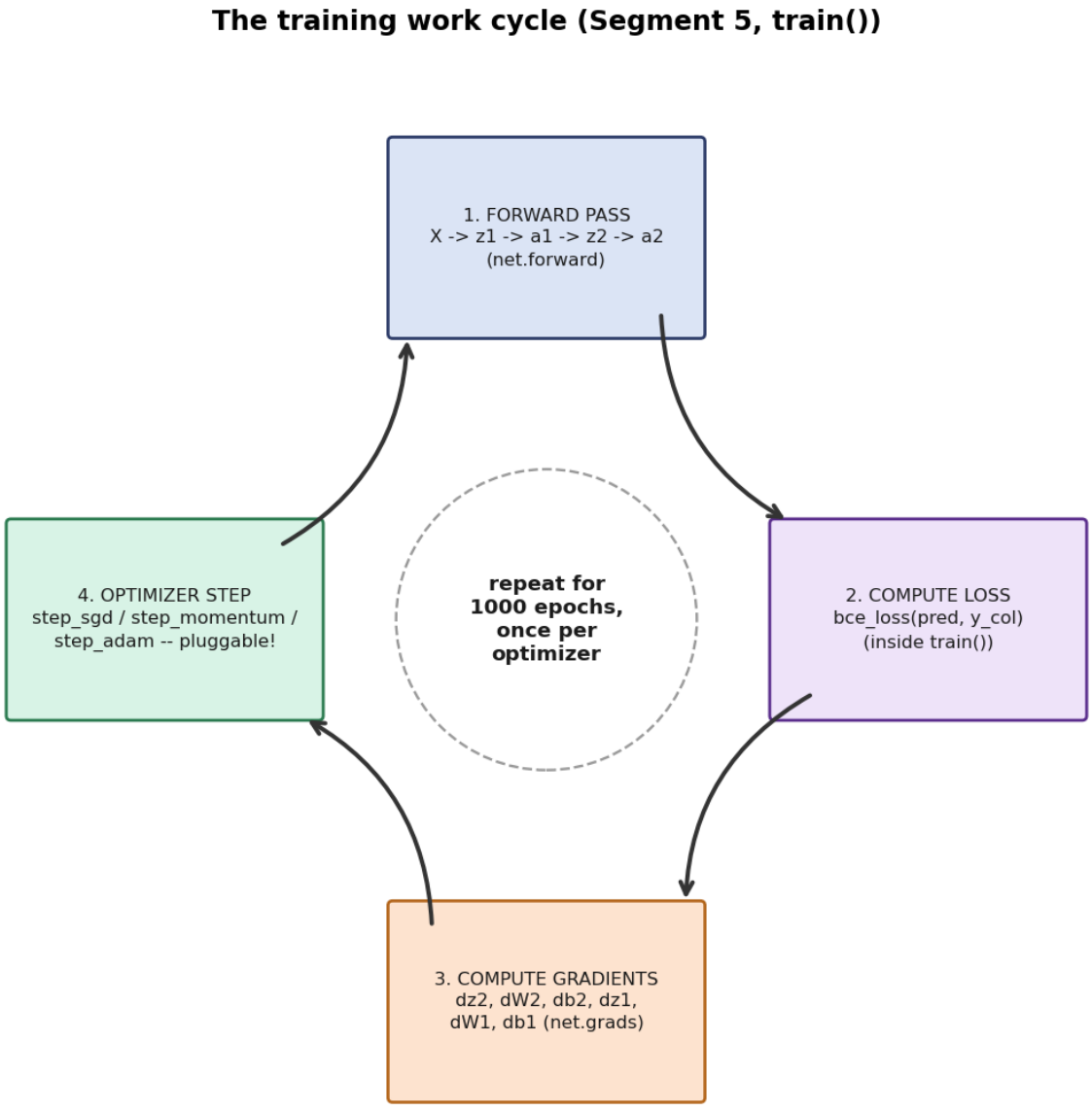
```
        final_pred = (net.forward(X) >= 0.5).astype(int).flatten()
        acc = (final_pred == y).mean()
        return losses, acc
```

Same accuracy-via-boolean-mean trick as Day 36: threshold at 0.5, cast True/False to 1/0, compare against the true labels, and average the matches into a single accuracy fraction.

```
configs = [("sgd", 1.0), ("momentum", 1.0), ("adam", 0.05)]
results = {}
for opt, lr in configs:
    losses, acc = train(X, y, opt, lr, epochs=1000)
    results[opt] = (losses, acc)
    fl, fa = losses[-1], acc
    print(f"{opt:10s} lr={lr}: final_loss={fl:.4f}  final_accuracy={fa:.3f}")
```

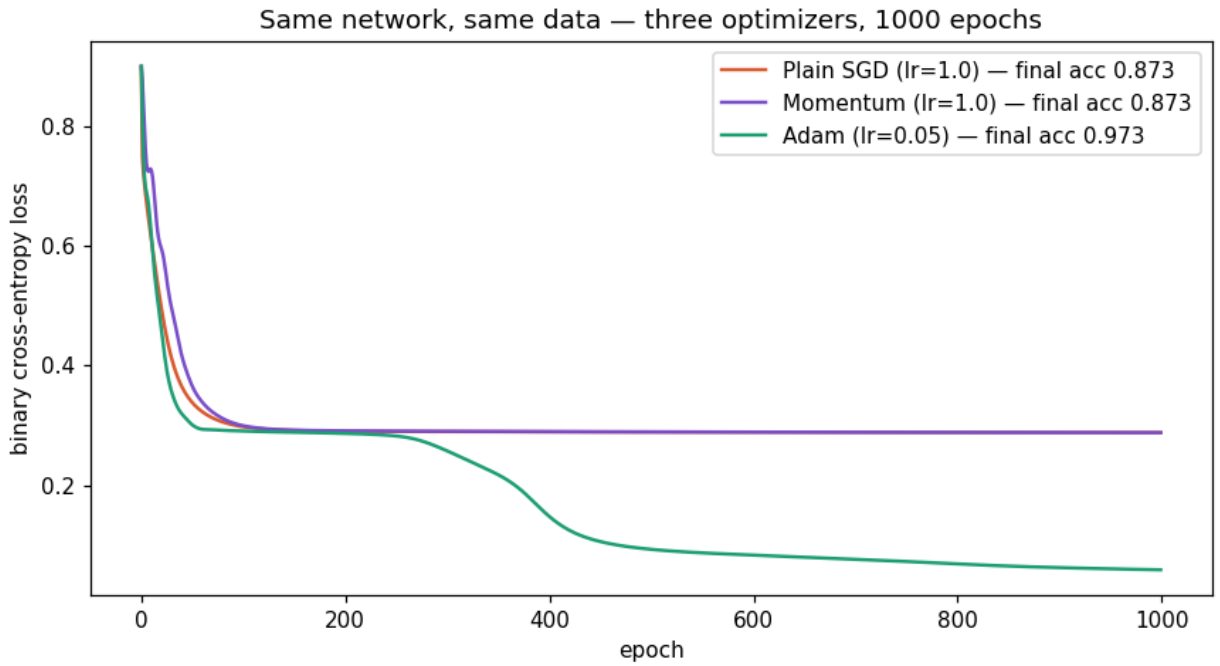
configs is a list of (optimizer name, learning rate) tuples — each optimizer gets its own individually tuned **lr**, unpacked directly in the **for opt, lr in configs:** loop header. **results[opt] = (losses, acc)** stores each run's full loss history and final accuracy in a dictionary keyed by optimizer name, so the plotting code afterward can look each one back up by name.

SYSTEM DIAGRAM



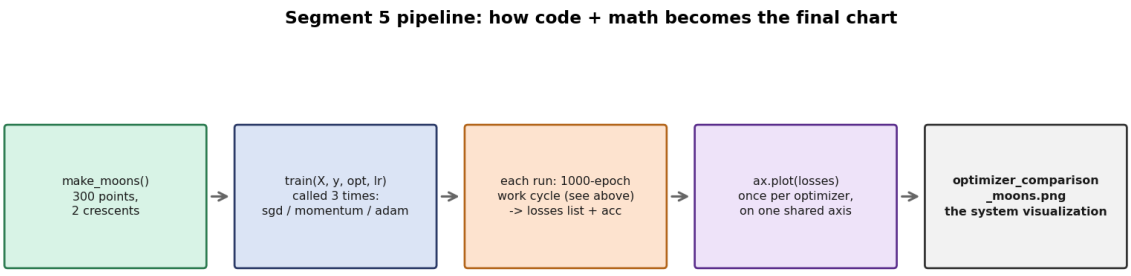
This is **train()**'s epoch loop as a work cycle — the same 4-stage shape as Day 36's, except stage 4 is now pluggable: which box actually runs depends on the **optimizer** string, but the forward pass, loss, and gradient computation stay identical no matter which optimizer is chosen.

REAL GRAPH (script's own output)



Not a mockup — this is **optimizer_comparison_moons.png**, this exact script rerun fresh: three calls to **train()**, three loss curves on one shared axis. SGD and Momentum both flatten out around loss 0.29 and stall there for the full 1000 epochs; Adam breaks through that same plateau. This is the opposite ranking from the toy surface above — the real, honest point of this whole script.

SYSTEM DIAGRAM



Zooming out: the full pipeline this segment runs end to end, from generating the dataset through three full training work cycles to the final comparison chart — every box here is code you've now read line by line.

Segment 6 — What This Maps to in PyTorch (lines 280–293)

```
        print("loss_fn = nn.BCELoss()")
        print("  # same formula as bce_loss() above")
        print("optimizer = torch.optim.SGD(params, lr=1.0, momentum=0.9)")
        print("  # step_momentum()")
        print("optimizer = torch.optim.Adam(params, lr=0.05)")
        print("  # step_adam()")
```

These are just **print()** calls containing plain text — not executable code, and not a comparison to real PyTorch output (PyTorch isn't imported anywhere in this file). In the actual script each statement and its # comment sit on one padded line — split across two lines here only so the page can show them without cutting text off. They're reference lines showing which PyTorch class corresponds to which hand-written function above, so the mapping is explicit before Day 38 potentially introduces the real library.

Segment 7 — main() and the Entry-Point Guard (lines 296–309)

```
def main():
    demo_loss_functions()
    demo_toy_surface_optimizers()
    demo_moons_optimizer_comparison()
    note_on_pytorch()
```

Same pattern as Day 36: one function calling every demo in the order they should run — the single source of truth for "what does running this script do."

```
if __name__ == "__main__":
    main()
```

The same entry-point guard as every script in this roadmap: __**name**__ only equals **"__main__"** when this file is run directly, so **main()** won't silently re-execute if this file is ever imported elsewhere (the way Day 36's **TinyNet** or **sigmoid** could be imported without triggering all its demos).

Quick Syntax Glossary — New or Reinforced Constructs

`x, y = xy` — Unpacking assignment: splits a 2-element array/tuple into two separate variables in one line.

`array[-1]` — Negative indexing: **-1** means the last element, **-2** the second-to-last, and so on.

`.copy()` — Makes an independent copy of an array. Without it, appending a reference to a list and later mutating the original would silently change the already-appended entry too.

`np.random.RandomState(seed)` — Creates an independent random generator, separate from NumPy's shared global random state — useful for guaranteeing two things start from the exact same 'randomness' regardless of what else has called **np.random** elsewhere.

`{k: expr for k in list}` — Dict comprehension — builds a dictionary in one line by evaluating **expr** once per item in **list**, using each item as the key.

`getattr(obj, name)` — Reads an attribute by its string name instead of hard-coding **obj.name** — lets code loop generically over a set of attributes.

`setattr(obj, name, value)` — Writes an attribute by its string name — the write counterpart to **getattr**.

`for k in some_dict:` — Iterating over a dictionary by default iterates over its **keys**.

`if/elif chain on a string` — Comparing a string variable against several literal values in sequence to branch behavior — Python has no switch statement, so this is the idiomatic replacement.

`list of tuples unpacked in a for loop` — **for a, b in [(x1,y1), (x2,y2)]:** unpacks each tuple into two loop variables per iteration, the same pattern used for (name, function) and (optimizer, lr) pairs throughout this roadmap.

End of Day 37 syntax reference. For the loss-function and optimizer concepts these lines implement, see the main Day 37 study guide PDF.